

BankBot – Selenium Java Automation Framework for Banking Application

Design Document

1. Project Overview

BankBot is a Selenium-based automation testing framework developed using Java, TestNG, and Maven to automate end-to-end functionalities of a banking application (Guru99 Demo site).

The framework is built using the **Page Object Model (POM)** design pattern, enabling:

- Reusability of components
- Scalable architecture
- Easy maintenance

It also supports data-driven testing, modular design, and automated reporting.

2. Objectives

The main objectives of this framework are:

- Automate core banking operations such as:
 - Login
 - Customer management
 - Account management
 - Fund transfers
 - Form validations
- Ensure test reliability using proper exception handling and wait mechanisms
- Capture screenshots automatically on failures
- Maintain clean, modular, and reusable code structure

3. Framework Architecture

Design Pattern

- Page Object Model (POM)

Separation of Responsibilities

- **Test Classes** → Handle business logic
- **Page Classes** → Manage UI interactions
- **Utility Classes** → Provide reusable helper methods

4. Project Structure

Bank_Bot/

```
| — src/test/java/
|   | — base/
|   |   | — BaseTest.java
|   |   | — BasePage.java
|   |
|   | — pages/
|   |   | — LoginPage.java
|   |   | — HomePage.java
|   |   | — NewCustomerPage.java
|   |   | — EditCustomerPage.java
|   |   | — NewAccountPage.java
|   |   | — EditAccountPage.java
|   |   | — FundTransferPage.java
|   |   | — DepositPage.java
|   |   | — BalanceEnquiryPage.java
|   |
|   | — tests/
|   |   | — LoginTest.java
|   |   | — CustomerTest.java
```

```
| | |— AccountTest.java
| | |— FundTransferTest.java
| | |— FormValidationTest.java
| |
| |— utils/
| | |— ConfigReader.java
| | |— DriverFactory.java
| | |— ScreenshotUtil.java
| |
| |— listeners/
| | |— TestListener.java
|
|— src/test/resources/
| |— config.properties
| |— testng.xml
|
|— screenshots/
|— pom.xml
|— README.md
```

5. Technologies Used

- Java
- Selenium WebDriver
- TestNG
- Maven
- Apache Commons IO
- Extent Reports (optional)

6. Key Components

BaseTest

- Initializes WebDriver
- Launches browser before execution
- Handles teardown after test completion

BasePage

Provides reusable methods such as:

- click()
- type()
- getText()

Also manages waits and handles stale element issues

Page Classes

Each page includes:

- Web element locators
- UI interaction methods
- Reusable functionality

Test Classes

- Define test scenarios
- Use page class methods
- Perform assertions for validation

Utility Classes

- **ConfigReader** → Reads configuration values

- **DriverFactory** → Manages browser setup
- **ScreenshotUtil** → Captures screenshots

Listener

- Implements ITestListener
- Automatically captures screenshots on failures

7. Test Modules

Module 1: Login

- Valid login
- Invalid login
- Blank input validation
- Logout

Module 2: Customer Management

- Add new customer
- Verify success messages
- Edit customer information
- Validate duplicate email

Module 3: Account Management

- Create account
- Verify account creation
- Edit account details
- Validate invalid customer ID

Module 4: Fund Transfer

- Transfer funds between accounts
- Verify balance updates
- Validate invalid payee scenarios

Module 5: Form Validation

- Empty field validation
- Non-numeric input validation
- Future date validation

8. Screenshot Handling

- Implemented using ITestListener
- Automatically captures screenshots on test failure
- Stored in: [/screenshots/](#)

9. How to Execute the Project

Using Maven

mvn clean test

Using TestNG

- Right-click on [testng.xml](#)
- Run as TestNG Suite

10. Reporting

- TestNG Reports: [/target/surefire-reports/](#)
- Extent Reports (optional): [/reports/](#)

11. Challenges Faced

- Handling browser alerts effectively
- Managing stale element exceptions
- Dealing with instability of Guru99 demo site
- Generating dynamic test data (emails, IDs)

12. Improvements Implemented

- Added retry mechanism for stale elements
- Used explicit waits for better stability
- Implemented dynamic test data generation
- Centralized configuration handling
- Enabled automatic screenshot capture on failure

14. Conclusion

This framework represents a robust and scalable automation solution with:

- Clean architecture
- Proper exception handling
- Maintainable code structure

It demonstrates an industry-level approach to building reliable Selenium automation frameworks.